

SMART TRAFFIC MANAGEMENT SYSTEM FOR URBAN CONGESTION CONTROL

Group No. 04

Course-Coordinator : Yogita Narule



Table of contents

01

Solution
Design

02

Tool
Justofication

03

Work Plan &
Time line

04

Work
distribution

05

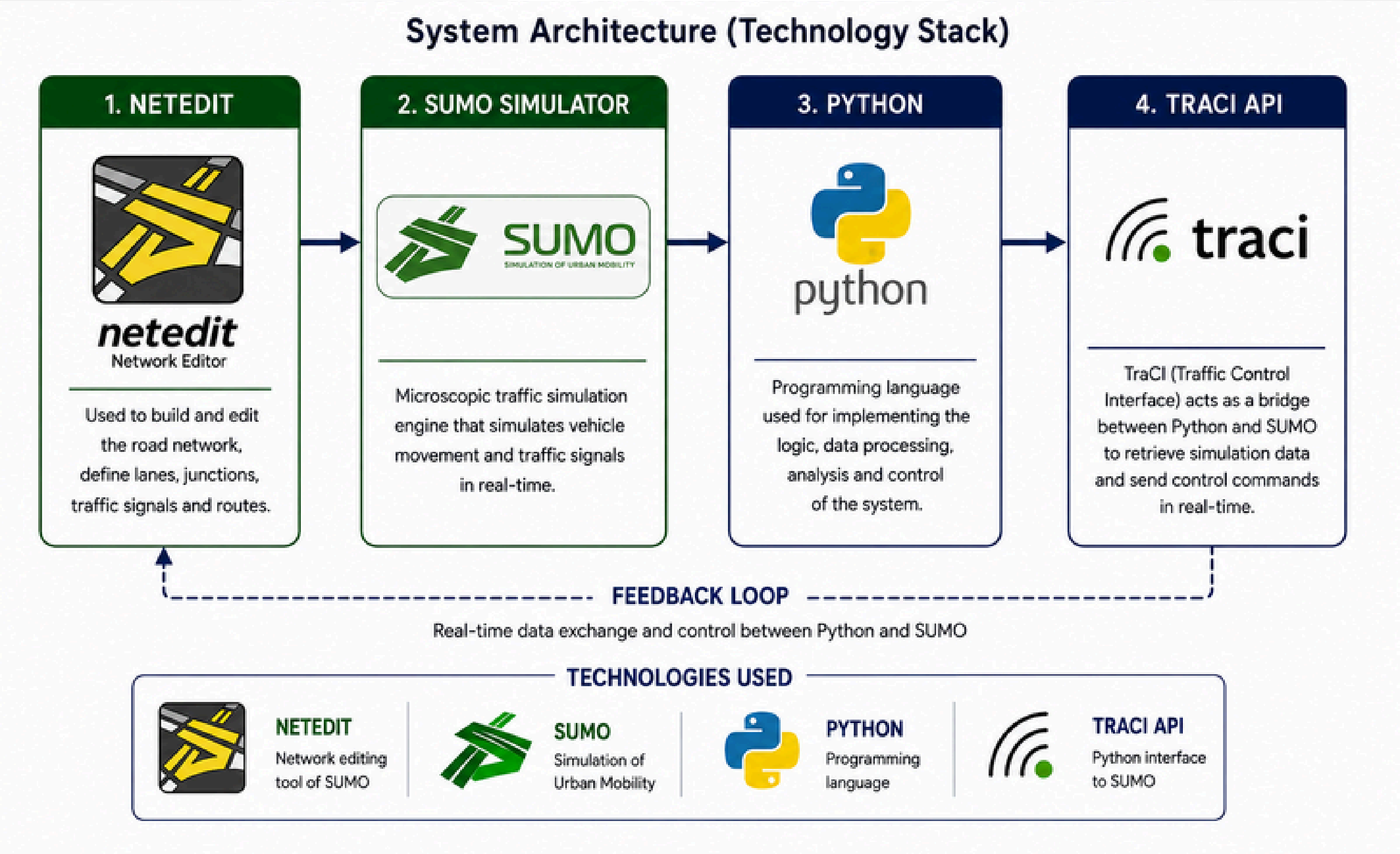
Risk &
Mitigation

06

Team
Coordination

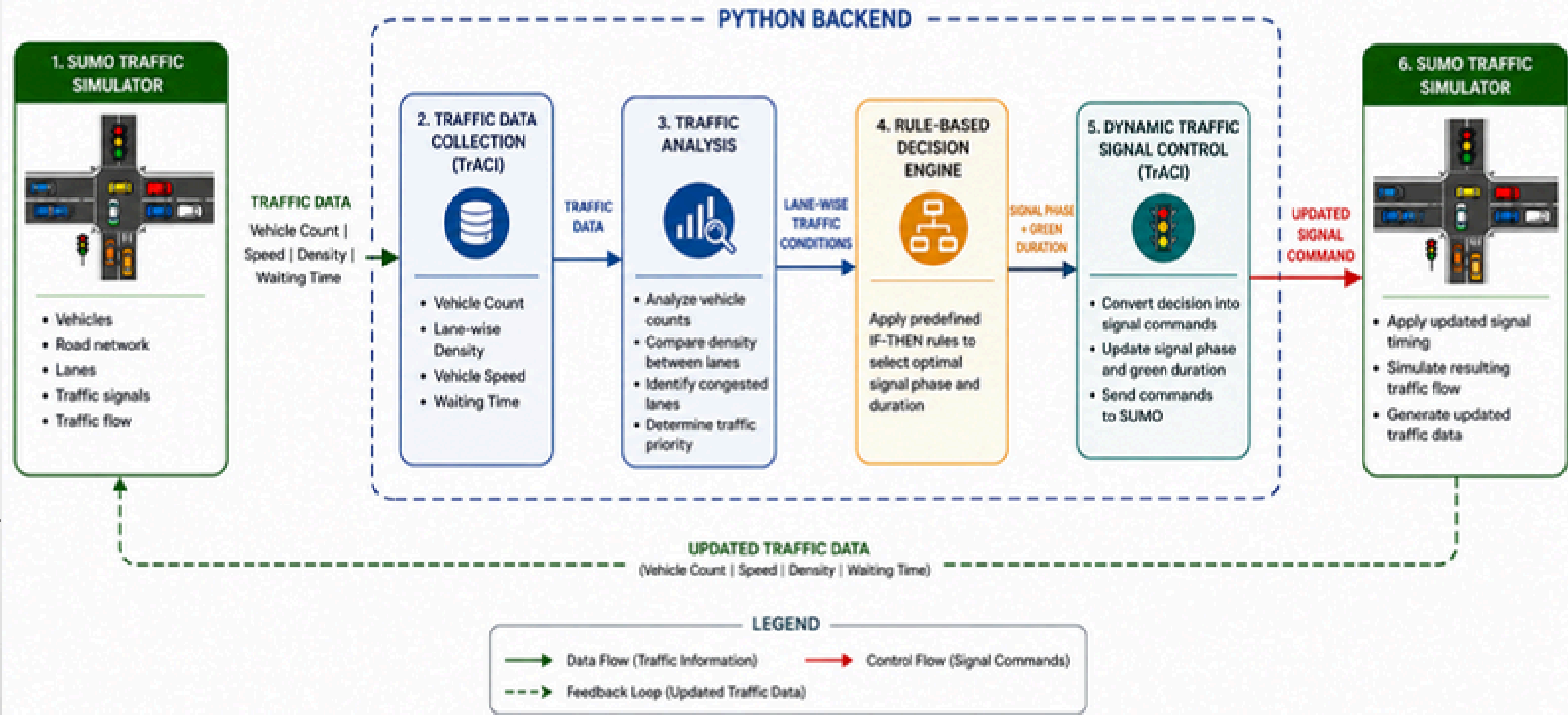


SYSTEM ARCHITECTURE DIAGRAM



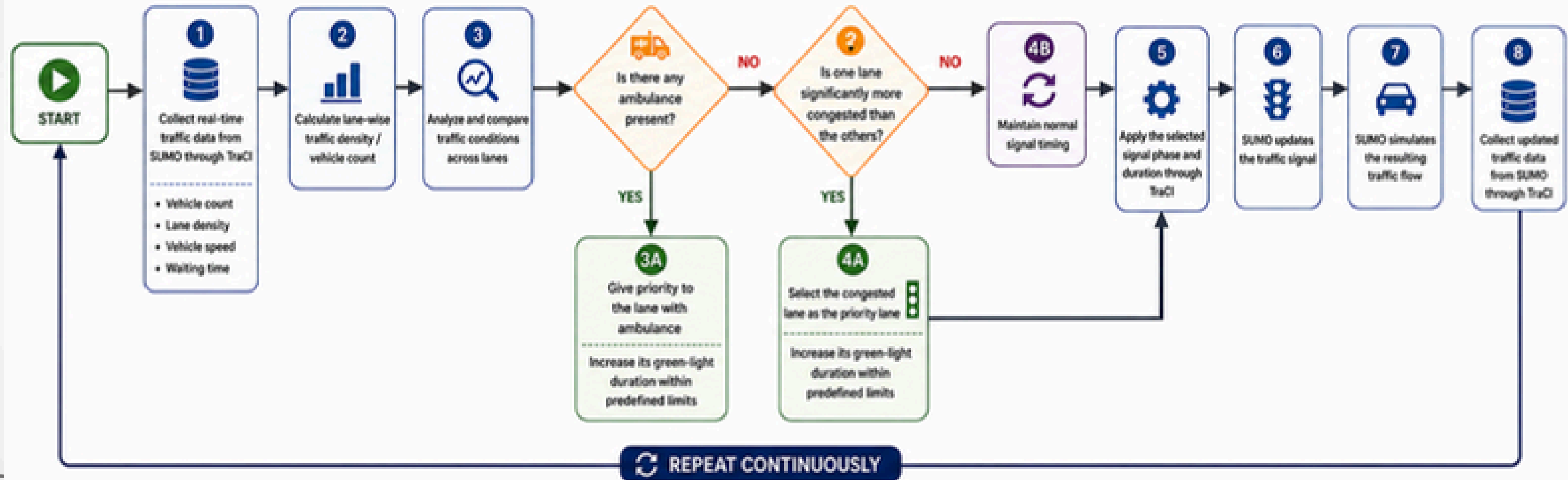
DATAFLOW DIAGRAM

SMART TRAFFIC MANAGEMENT SYSTEM LEVEL-1 DATA FLOW DIAGRAM (DFD)



LOGIC DIAGRAM

RULE-BASED TRAFFIC SIGNAL CONTROL LOGIC FLOWCHART



RULE-BASED DECISION RULES



1. HIGHER LANE TRAFFIC

→ Higher green-time priority



2. BALANCED TRAFFIC

→ Normal signal cycle



3. REDUCED CONGESTION

→ Restore normal timing



4. GREEN TIME LIMITS

→ Green time must remain within predefined minimum and maximum limits



TOOL JUSTIFICATION



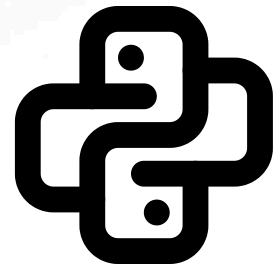
SUMO & NetEdit: Simulator Editor



- **Free & Open-Source:** SUMO is free to use, so you don't need to pay expensive fees like you might for VISSIM or AIMSUN.
- **Live AI Connection:** Using TraCI, SUMO can communicate with your Python program while the simulation is running.
- **Microscopic Precision:** SUMO shows each vehicle separately. So, you can know whether a vehicle is a car, bike, truck, or ambulance and control traffic accordingly.
- **Built-in Road Builder:** SUMO is just a visual tool with NETedit where we can draw roads, lanes, and intersections easily without creating everything manually in code.

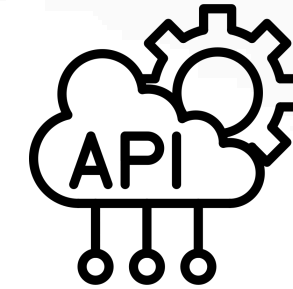


TOOL JUSTIFICATION



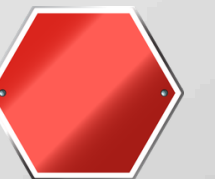
PYTHON

- **Easy to use:** Python has simple syntax, so traffic-control rules can be developed and modified easily.
- **Works with SUMO:** Python connects with SUMO using the TraCI API, allowing real-time vehicle data and traffic-signal control.
- **Future AI integration:** Python has strong support for AI/ML and data-processing, making it suitable if we upgrade our rule-based system to machine learning later.



TRACI API

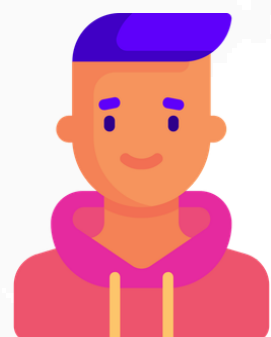
- **Real-time communication:** TraCI acts as a bridge between SUMO and Python, allowing Python to receive live vehicle information from SUMO.
- **Control traffic signals:** Python can send commands back through TraCI to change traffic lights dynamically based on vehicle conditions.
- **Alternative option:** Instead of TraCI, **libsumo** can be used. It provides SUMO's functionality directly within the application, reducing the need for separate client-server communication.



[illegible]



TASK DISTRIBUTION



Member 1

Frontend & Environment :

SUMO Netedit – road &
lane creation , Traffic
environment setup ,
Different signal formation
(4 lane , 3 lane)



Member 2 & 3

Backend Logic :

Python scripting , TraCI
API integration , AI rule-
based creation Traffic
signal control logic



Member 4

Documentation :

Team Co-ordination,
Design documents
Testing reports ,
Presentation creation,
Research analysis



RISK PLANNING & MITIGATION

SUMO-Python Connection Failure

Impact : Signal control will not work

Mitigation : Test the TraCI connection separately before full integration

Incorrect Vehicle Detection

Impact : Wrong traffic signal decision

Mitigation : Verify vehicle count and vehicle type data in SUMO

Emergency Vehicle Priority Failure

Impact : Ambulance may face unnecessary delay

Mitigation : Create and test a separate emergency-priority rule





TEAM COORDINATION



GitHub
Repository



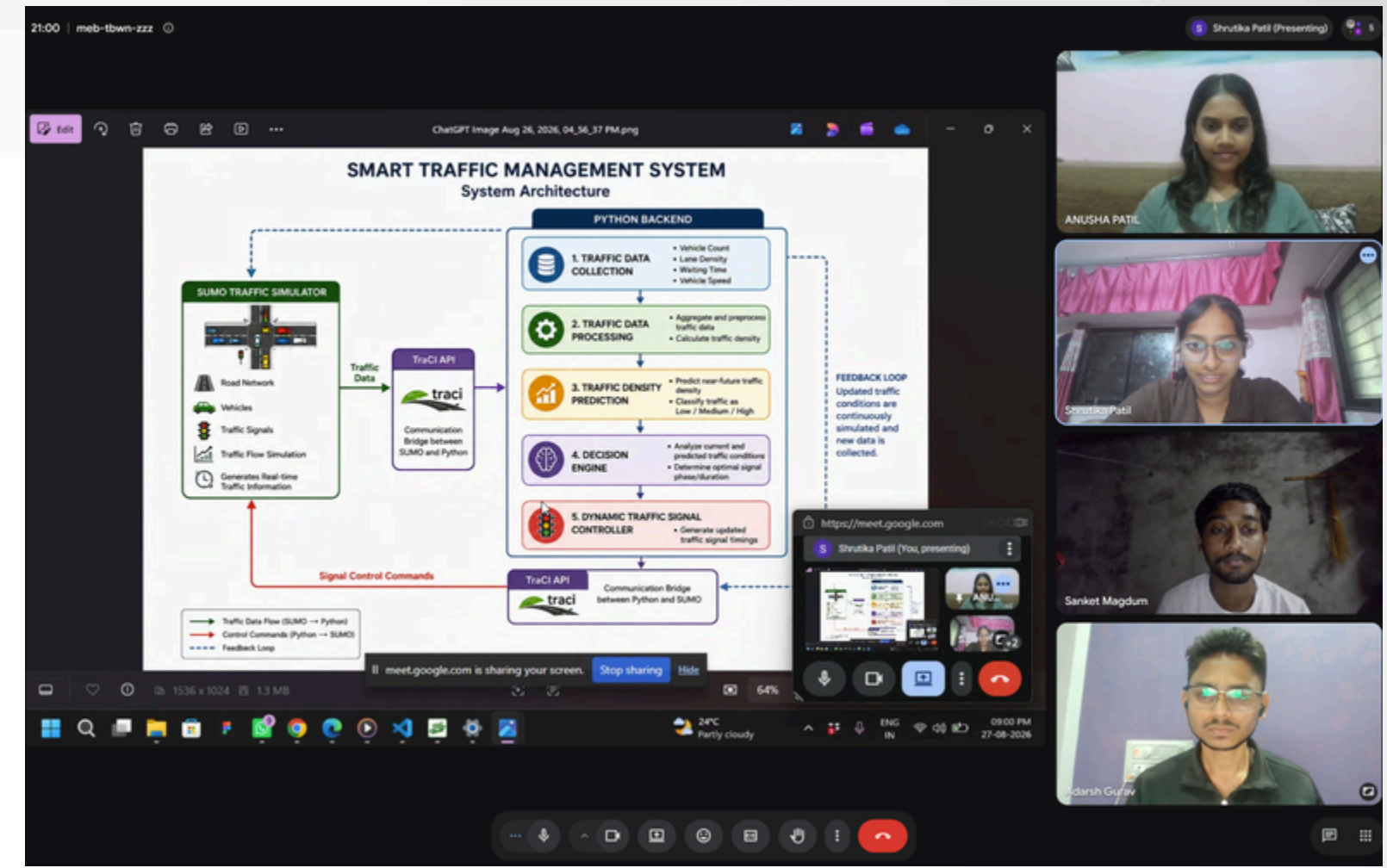
Group
Discussion



Regular Team
Meetings



Task
Sharing



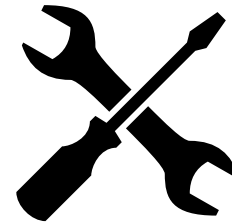


CONCLUSION & NEXT STEPS



Design Complete

System architecture and workflow are finalized.



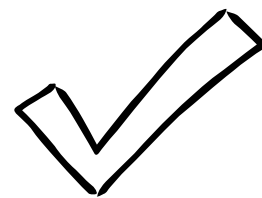
Tools Validated

SUMO, Python, TraCI are selected and validated.



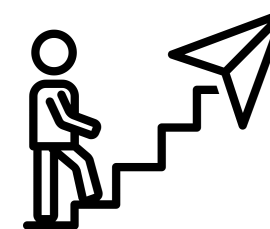
Risks Planned

Key risks have been identified with mitigation strategies.



Task Done

SUMO Setup and frontend visualization Done.
Working on Python AI logic and TraCI integration.



Next Step

Complete integration, testing, and final system implementation.



Thanks

**The Journey Towards Smarter Traffic
Management Has Just Begun.
Stay Tuned!**

